

GPGPU-Parallel Re-indexing of Triangle Meshes with Duplicate-Vertex and Unused-Vertex Removal

Ingo Wald

NVIDIA

Abstract

We describe a simple yet highly parallel method for re-indexing “indexed” data sets like triangle meshes or unstructured-mesh data sets—which is useful for operations such as removing duplicate or un-used vertices, merging different meshes, etc. In particular, our method is parallel and GPU-friendly in the sense that it all its steps are either trivially parallel, or use GPU-parallel primitives like sorting, prefix-sum; thus making it well suited for highly parallel architectures like GPUs.

1. Introduction

Many applications and algorithms operate on some form of “indexed” data structures like triangle or general polygon meshes, tetrahedral or general unstructured meshes, etc. In these data structures, a set of “vertices” (usually stored in a single contiguous array) can be shared among multiple different “elements” (such as triangles or tetrahedra) by having the latter be defined through sets of integer indices that reference elements in the vertex array.

It is often desirable for such mesh data structures to not contain either un-used or duplicate vertices; however, many operations that one might perform on a mesh—such as building a mesh from individual elements, merging or splitting meshes, computing sub-sets of a mesh, etc—are most easily implemented in a way that produces just such vertices.

Thus, it is often required to remove such duplicate and/or un-used vertices, which is often called “re-indexing” or “re-meshing”. Conceptually, this is most easily done by starting with a new empty mesh, adding vertices as required, and keeping track (e.g., using a `std::map`) of which vertices have already been added to the mesh, and with which index. E.g., for a triangle mesh, this could look roughly like this:

```
Mesh remesh(Mesh oldMesh)
{
    Mesh newMesh = empty mesh;
    std::map<vertex,int> alreadyInserted;
    for (triangle in oldMesh)
        for (each vertex v in triangle)
            if (alreadyInserted
                .alreadyContains(v)) :
                v_idx = alreadyInserted[v];
            else
                v_idx = newMesh.vtx.size()
                newMesh.vtx.push_back(v)
                alreadyInserted[v] = v_idx;
    ....
}
```

In particular since the appearance of the Standard Template Library (STL) this solution is trivially simple to code, and at least for small meshes works quite well. However, there are two issues with this approach: First, it is intrinsically serial, and even on a CPU can quickly become a bottleneck once meshes become non-trivially small. And second, this serial nature makes it manifestly unsuitable to use on GPUs.

In this article, we describe a GPGPU-parallel method for building a duplicate- and unused vertex-free mesh from a given input mesh. With “GPGPU”-parallel we mean that our method can be expressed solely through a sequence of operations that are either trivially parallelizable (such as setting all elements of an array to a certain value), or are operations that are readily available in parallel form, such as parallel sorting or parallel prefix sums in libraries such as `thrust` [BH11] or Threading Building Blocks (TBB) [Wik]. We walk the user through our algorithm using a simple example, but also provide reference source code for both GPU (using `thrust` and `CUDA` [NBGS08]), and CPU (using `TBB`).

The resulting algorithm is simple and parallel, works well on both CPUs and GPUs, and is simple enough for us to be certain that others must have invented and used it before, many times. However, in absence of any easily findable yet detailed description of this method we ourselves have ended up repeatedly re-inventing this method for different applications. The goal of this article is to save others this effort, and to provide an easy-to-find reference description (and implementation) of this useful method.

2. The Method

In this section we will walk the reader through our method in a step by step fashion; to better illustrate how these steps might affect a given input mesh we will be illustrating each step using a simple test case that contains two duplicate and two un-used vertices. In

particular, we will be using the following triangle mesh with named vertices A, B, . . . , and “triangles” of three vertex indices each:

```
> vtx = {A,B,C',X,D',C'',E,F,Y,D"}
> idx = {(0,1,2) (0,2,4) (5,6,7) (5,7,9)}
```

Note for that this example it does not matter whether the vertices are 2, 3, or any other dimensional vertices (or what other type of data, for that matter); however, we do assume for this paper that vertices with the same alphabetical letter are exact duplicates of each other (the primes and quotes just illustrate different duplicates of a given vertex), and that vertices with different alphabetical letters are different. We also assume that there is a obvious way of ordering two vertices as to which one is “less” than another.

In our sample mesh, the vertices X and Y are not used by any triangle; the vertices C and D are each stored twice in different locations—to show how these move during our algorithm we have intentionally tagged these with either one or two primes; however, the value of C' and C"—respectively D' and D"—are considered to be the same. The actual four triangles formed by this mesh are, obviously, (A, B, C), (A, C, D), (C, E, F), and (C, F, D).

2.1. Step 1: Replace un-used vertices w/ any used one

The core goal of our method (which we describe below) is not aimed at removing *unused* vertices, but only at removing *duplicate* vertices. However, we can make it solve the problem of unused vertices, too, by simply replacing each unused vertex with any other used one (e.g., the first vertex of the first triangle): this transforms unused vertices into duplicate ones, and though after this step these now-duplicate vertices are still not referenced by any triangle they will in the later stages be merged with the used vertex whose value they were overwritten with, thus effectively removing them.

To do this replacing of unused vertices with a used one we first allocate a temporary array `isUsed[]` with one bool per vertex, and initialize this to false for each vertex. Second, we parallel-iterate over each vertex index used in any of the mesh’s index elements (i.e., each vertex index of each triangle), and mark that index’s corresponding array element as used:

```
parallel_for(each vertex index vtxID)
    isUsed[vtxID] = true;
```

This operation is obviously parallel and can be trivially implemented in CUDA, using TBB, or through some `thrust::scatter`, etc. For our test case, this produces

```
: vtx = {A,B,C',X,D',C'',E,F,Y,D"}
> isUsed = {1,1,1, 0,1, 1, 1,1,0,1 }
```

(to improve readability we will, from here on, annotate each step’s *input* arrays with a colon (:), and outputs from this step with a greater-than sign (>).

Finally, we execute another parallel for over all vertices, check if the vertex is marked as used, and if not, overwrite it with the first triangle’s first vertex:

```
parallel_for(i = 0 ... numVertices)
    if (!isUsed[i])
        vtx[i] = vtx[triangle[0].vtxIdx[0]]
```

For our sample mesh, this evaluates to

```
# before overwriting unused vertices
: vtx = {A,B,C',X,D',C'',E,F,Y,D"}
: isUsed = {1,1,1, 0,1, 1, 1,1,0,1 }
# after overwriting unused vertices
> vtx = {A,B,C',A',D',C'',E,F,A'',D"}
```

which no longer contains the previously unused vertices X and Y, but instead use two new copies of the used vertex A.

2.2. Step 2: Make each reference to a duplicated vertex point to the same instance of that vertex

In this step, we want to ensure that for any duplicate vertex (e.g., C being duplicated into C', C'') all indices referring to the different copies of this vertex will get replaced with a single index to only a single one of those copies (all other copies will remain in the vertex array for now, but have no indices pointing to them any more). Whether all point to C' or C'' does not matter, as long as all point to the same.

To do this, we will first sort all vertices, which will then allow us to spot duplicate vertices by simply comparing them to their predecessor. This sorting will change the order of the vertices; so we first need to ensure that we can later undo this re-ordering. To do this, we take three steps: First, we create a copy of our (cleaned) vertex array `oldVtx = vtx`. Second, we create an array `orgID[]` with one int per vertex, and fill that with the identity (for example, using `thrust::sequence`):

```
> orgID = { 0,1,2,... }
```

Using this helper array, we now perform a key-value sort with the vertex array being the key, and the ID-array being the value. Key-value sorts are commonly available in different parallel programming libraries, for example via `thrust::key_value_sort()` in `thrust`, or via `tbb::parallel_sort()` with a `tbb::zip_iterator` in TBB. For our sample data, this produces

```
# before sorting
: vtx = {A,B,C',A,D',C'',E,F,A,D"}
: orgID = {0,1,2, 3,4, 5, 5,6,8,9}
# after key-value sort
> vtx = {A, A, A, B, C, C',D',D'',E, F}
> orgID = {0, 3, 8, 1, 5, 2, 4, 9, 5, 6}
```

Note the ordering of C' vs C'' (or D' vs D'') is completely arbitrary, since both C' and C'' are the same value - we just tagged one with a prime to show it has a different index, but they are indistinguishable for the sort operation, so for an “unstable” sort algorithm may end up in any order. Since the `orgID` array has undergone exactly the same permutation as the vertices we will later on be able to use this to reverse this sort operation (Section 2.4).

Using this sorted array, we can now easily define which vertices are duplicates of another same vertex with lower index, and which ones are the respectively first occurrence of a given vertex. To do this we compute an array `nodup[]` (with one bool per vertex), and execute a parallel kernel that sets each element entry to true if it is either the first vertex in the list, or different from its predecessor:

```
parallel for i = 0..numVertices
    nodup[i] = (i == 0) || (vtx[i] != vtx[i-1])
```

For our example:

```
: vtx    = {A, A, A, B, C", C', D', D", E, F}
> nodup = {1, 0, 0, 1, 1, 0, 1, 0, 1, 1}
```

Next, we can compute a new array index `newIdx` as a postfix sum over `nodup`:

```
: nodup = {1, 0, 0, 1, 1, 0, 1, 0, 1, 1}
> newIdx = {1, 1, 1, 2, 3, 3, 4, 4, 5, 6}
```

then subtract one from each element of this array:

```
> newIdx = {0, 0, 0, 1, 2, 2, 3, 3, 4, 5}
```

Again, postfix sums are available in common parallel programming packages, e.g., via `thrust::inclusive_scan`; to then subtract one we use a trivially simple CUDA kernel[†]. This array now gives, for each vertex, a new unique array index for an array that would not contain duplicates.

2.3. Step 3: Computing the New Vertex Array

In the previous section's `newIdx` array, the least index by design must be the highest vertex index, so the number of unique elements in our target vertex array is the last element plus one:

```
> newN = newIdx.back() + 1 = 6
```

We can now allocate a new array of vertices with `newN` entries; then, for each index $j < N$ in the original array of vertices, read the vertex at position j in the original vertex array, and write it to position `newIdx[j]` in the new vertex array.

```
parallel for i=0..numVertices
  if nodup[i] // < this is optional
    newVtx[newIdx[i]] = vertex[i]
```

This operation is identical to a `scatter` operation, so where available can also be implemented as such (e.g., via `thrust::scatter`).

For our sample data, this produces:

```
> newVtx = { A, B, C", D', E, F }
```

(again, it does not matter which version of C' / C'' or D' / D'' were written, they have the same value).

2.4. Step 4: Computing the New Index Array

The previous step has computed a new and compact array of vertices that no longer contains duplicate vertices; however, this—and the preceding sort operation—have necessarily moved vertices from their original array locations, so the old vertex indices are no longer valid, and must be updated.

To do this, we first create an array `perm` of one int per original input vertex, and for each element set `perm[orgID[j]]` to j .

```
parallel_for(i=0..numVertices)
  perm[orgIdx[i]] = i
```

(again, this could be done with a `thrust::scatter` operation). After this operation the `perm` array effectively describes the permutation done by sorting. For our example:

```
: orgID = {0, 3, 8, 1, 5, 2, 4, 9, 5, 6}
> perm  = {0, 3, 5, 1, 6, 4, 8, 9, 2, 7}
```

With this, we can now update every old index i to the value `newIdx[perm[i]]`:

```
: idx(old) = { (0,1,2) (0,2,4) (5,6,7) (5,7,9) }
: perm     = {0, 3, 5, 1, 6, 4, 8, 9, 2, 7}
: newIdx    = {0, 0, 0, 1, 2, 2, 3, 3, 4, 5}
> idx(new) = { (0,1,2) (0,2,3) (2,4,5) (2,5,3) }
```

To explain why this works: The first lookup ($i \rightarrow \text{perm}[i]$) translates from the original vertex index i to where this vertex would have been after sorting; the second ($\text{perm}[i] \rightarrow \text{newIdx}[\text{perm}[i]]$) translates from the position after sorting to the position after compacting (ie, after removing the duplicates)—which is where the vertex at i has ended up after both steps.

2.5. Checking our Example

To visually show that this is correct for our example, let's first look at the original mesh with duplicates:

```
: vtx(old) = {A, B, C', X, D', C", E, F, Y, D"}
: idx(old) = { (0,1,2) (0,2,4) (5,6,7) (5,7,9) }
> tris(old) = { (A, B, C') (A, C', D') (C", E, F) (C", F, D") }
```

Then, for reference, the new mesh looks as follows:

```
: newVtx = { A, B, C", D', E, F }
: newIdx = { (0,1,2) (0,2,3) (2,4,5) (2,5,3) }
> tris    = { (A, B, C) (A, C", D') (C", E, F) (C", F, D') }
```

These are indeed the same triangles with the same vertices, except for the arbitrary choice of which copy of a vertex may have been chosen for duplicate vertices (keep in mind that C' and C'' —and D' and D'' —are actually the same vertices, with the prime only used for illustration of the data movement).

3. Sample Use Cases

Though there are probably more uses for this “re-indexing” of meshes, in practice we have found this technique useful in particular for the following tasks:

3.1. Removing duplicate vertices

Sometimes meshes can contain duplicate vertices, either because whoever created the meshes did not care about removing them, or because vertices contained some attributes that made them different (e.g., vertex colors) that the application then discarded. Simply running the above method will create a new mesh that has the same triangles, but has all duplicate vertices removed.

3.2. Merging meshes that share (some) vertices

This is a special case of the previous one: to merge two (or more) individual meshes, one can in a first step just concatenate the different meshes' vertex arrays, then increase all of the second mesh's

[†] This subtracting of one could also be avoided entirely by setting the first element in the `nodup` array to 0 instead of 1, but we chose not to do this for didactic reasons

N	8	64	1024	4096	8192
mesh: number of quads (in)	64	4.10K	1.05M	16.78M	67.11M
mesh: number of vertices (in)	320	20.48K	5.24M	83.89M	335.54M
mesh: number of vertices (out)	81	4.22K	1.05M	16.79M	67.13M
time (CPU, serial)	<1ms	1.1ms	854ms	17.6s	78s
time (CPU, parallel w/TBB)	<1ms	1.6ms	94ms	1.5s	6.2s
time (GPU, thrust+CUDA)	<1ms	<1ms	9.2ms	92ms	338ms

Table 1: Evaluation of our method—comparing both scalar CPU reference, our parallel method on CPU, and a CUDA implementation thereof—for various different mesh sizes.

vertex indices by a value that equals how many vertices the first mesh originally contained, and append all those increased indices to the index array, etc. This operation is cheap and trivially parallelizable, but ends up with potentially duplicate vertices that the above method can then be used to remove.

3.3. “Soup to Mesh” conversion

If the input is a set of individual “fat” triangles (v_0, v_1, v_2) , (v_3, v_4, v_5) , (v_6, v_7, v_8) etc, then the above can be used to create a mesh representation by first creating a “dummy” mesh with $vtx=\{v_0, v_1, \dots\}$ and $idx=\{(0, 1, 2) (3, 4, 5), (6, 7, 8) \dots\}$; then using our algorithm to remove duplicates—the result is a valid and compact mesh representation.

3.4. Computing a subset of a mesh (e.g., splitting a mesh into sub-meshes)

Some operations require creating a mesh from a subset of triangles of another mesh, such as the set of all those triangles that share a given material type[‡]. Other examples are computing a compact mesh of only outer faces of a tetrahedral mesh, or a new unstructured mesh that only contains the tetrahedra from a mixed-element mesh, etc.

To create a compact mesh of only a subset of elements, one can simply create a new mesh that contains a copy of the full vertex array, but only those elements that one wants in this mesh; the above method will then remove all unused vertices and return a new, compact mesh with only used vertices.

3.5. Reference Code and Performance

Though we have previously used this method in many different contexts, for the purpose of some at least rough evaluation we also implemented some reference code that we have made available via MIT license under <https://github.com/ingowald/sampleCode-parallel-mesh-reindexing>. For this reference code we chose `float2` vertices and `int4` indices because such meshes were considered the easiest to

generate test cases for (see below); adapting this to other vertex and index formats should be trivial.

To evaluate the impact of parallelization we implemented three methods: a serial CPU reference method uses a `std::map` to track which vertices have already been added; a parallel version of our method on the host that uses Threading Building Blocks (TBB [Wik]; using the `tbb-dev` package on Ubuntu 20.04) for parallelization and parallel sorting; and a GPU-parallel method that uses a combination of thrust [BH11] (for sorting, prefix sum, etc) with small CUDA kernels for computing some of the other values. All three methods have been implemented as we would have used them in our own codes; careful tuning might give additional speedups, but this we believe to be true for each one of the three methods, so their relative performance should be roughly representative. All experiments are run on a desktop PC with a Intel Core i7-7820X CPU (8 physical cores at 3.6 GHz/4.3 GHz in regular/turbo mode). The GPU we used was an RTX 3090 “Founders Edition” GPU with 10,496 CUDA cores at 1.7 GHz, and 24 GB GDDR6X memory.

To evaluate scalability across different mesh sizes we implemented a generator that would, for a given N , generate a regular mesh of $N \times N$ quadrilaterals (i.e., $(N + 1) \times (N + 1)$ unique vertices) for which the vertices are at the same location as those of its neighbor quads, but where these vertices are nonetheless replicated. To also include some unused vertices we also place one unused vertex into the center of each quad, so every fifth vertex in the vertex array is unused.

The results of this evaluation—for various values of N —are given in Table 1. As can be seen from this, for smaller mesh sizes there is no major difference between the three methods; however, for larger meshes parallelization becomes very useful even on the CPU.

3.6. Summary and Discussion

We have described a simple—and in particular, parallel—method for re-indexing triangle meshes that removes duplicate vertices, and can also be used for remove unused vertices, for computing compact sub-sets of larger meshes, etc. In particular, the algorithm is well suited to GPU execution because all it requires are some trivially parallel kernels for reading from and writing to arrays, as well as parallel sort and parallel prefix sum that commonly available libraries like thrust already provide.

Though we have illustrated this algorithm only for triangle

[‡] One common example is the wavefront OBJ format, where one typically wants to split the input based on group or material type, but where all vertex indices are specified relative to a single global vertex array.

meshes, it works exactly the same for any other “indexed” representation such as quad meshes, general polygon meshes, unstructured tetrahedral, hexahedral, etc meshes, etc. And though its main use case obviously is to enable fast re-indexing on GPUs (for which a serial solution would not make any sense whatsoever) at least for non-trivial mesh sizes it is much more efficient than the serial variant even on the CPUs.

Arguably the biggest shortcoming of this method is that it requires two copies of both vertex and index array, plus some additional (though smaller) helper arrays, plus whatever temporary memory the sorting algorithm may require (which can again be as much as the data that is being sorted); which for really large meshes may become an issue.

References

- [BH11] BELL N., HOBEROCK J.: Thrust: A Productivity-Oriented Library for CUDA. In *GPU Computing Gems (Jade Edition)*, Hwu W. W., (Ed.). Elsevier, 2011. 1, 4
- [NBGS08] NICKOLLS J., BUCK I., GARLAND M., SKADRON K.: Scalable parallel programming with cuda, 2008. 1
- [Wik] WIKIPEDIA: Threading Building Blocks.
https://en.wikipedia.org/wiki/Threading_Building_Blocks.
1, 4, 5